AIMERZ.ai
Aim.Act.Achieve

NOTES

Serilization and De-Serilization

Externalizable in Java
Java Transient Keyword
Garbage Collection in Java

Serialization and De-Serialization in Java

Serialization and **Deserialization** are two important concepts in Java, especially useful for saving the state of an object and transferring it over networks or storing it in files. Let's discuss them with simple explanations, real-world analogies, and examples.

What is Serialization?

Serialization is the process of converting an object in Java into a sequence of bytes (a binary format). This makes it easy to save the object's data to a file or send it over a network.

Detailed Explanation: Java has a feature called **object serialization**. This lets us convert an object into a sequence of bytes. These bytes contain both the object's data and information about the object's type and the types of data it holds.

Once an object is serialized (converted into bytes), we can save it to a file. Later, we can read it back from the file and **deserialize** it, which means converting the bytes back into the original object in memory.

The best part is that this process works across different platforms. An object can be serialized on one computer and deserialized on a completely different computer, even if they use different operating systems.

Real-World Analogy -> **Physical Document:** Think of serialization as taking a real object, like a physical document, and converting it into a digital form (like a PDF). Once in digital form (Converted to PDF), it's easier to share, store, or send via email.

Why Use Serialization?

Storage: We can save an object's state to a file or database.

Communication: We can send the object over a network to another program.

How Serialization work in Java

In Java, to make an object serializable, the class of that object needs to implement the Serializable interface. This interface doesn't contain any methods but serves as a marker to tell the Java compiler that this class can be serialized. And it is also called marshalling.

To serialize an object, we use the writeObject() method from the ObjectOutputStream class. The object must also implement the Serializable interface to be serialized.

java.io.Serializable interface

The `java.io.Serializable` interface is a marker interface, meaning it doesn't have any methods or data members. It is used to "mark" Java classes, allowing their objects to gain a specific capability. Other marker interfaces include `Cloneable` and `Remote`.

A class must implement the `Serializable` interface if its objects need to be saved (persisted).

By default, the `String` class and all wrapper classes implement the `java.io.Serializable` interface.

Example

Let's create a class called `Person` with attributes `name` and `age`.

```Java
import java.io.Serializable;

class Person implements Serializable {
      private String name;
      private int age;

      // Constructor
```

```
        public Person(String name, int age) {
        this.name = name;
        this.age = age;
        }

        // Getters
        public String getName() {
        return name;
        }
        public int getAge() {
        return age;
        }
}
```

In the above example, *Person* class implements Serializable interface. Now its objects can be converted into byte stream. The main class implementation of is showed in the next code.

FileOutputStream class

Purpose: FileOutputStream is used to write data to a file.

How it works: It writes data byte by byte to the file, so it's often used for binary data like images, audio files, or any file that needs to be saved exactly as it is.

Example: If you want to save text or data to a file, you can use FileOutputStream to create the file and write data into it.

This will be explained in full in **IO Concepts...**

ObjectOutputStream class

The `ObjectOutputStream` class is used to save (write) data and Java objects to an output stream, such as a file or network connection. It

can handle basic data types (like integers, floats, etc.) as well as complex objects.

However, only objects that implement the `java.io.Serializable` interface can be written to an `ObjectOutputStream`. This means the class of the object needs to be "serializable" so that it can be converted into a byte stream and saved.

Constructor

1) public ObjectOutputStream(OutputStream out) throws IOException {}	It creates an ObjectOutputStream that writes to the specified OutputStream.

Important Methods of ObjectOutputStream class for Serialization

Method	Description
1) public final void writeObject(Object obj) throws IOException {}	It writes the specified object to the ObjectOutputStream.
2) public void flush() throws IOException {}	It flushes the current output stream.
3) public void close() throws IOException {}	It closes the current output stream.

Serialization Example Code:s

Serializing an instance of Person by writing it to a file:

```Java
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;

public class SerializeExample {
       public static void main(String[] args) throws Exception {
       Person person = new Person("Alice", 30);

       FileOutputStream fileOut = new FileOutputStream("person.ser");
       ObjectOutputStream out = new ObjectOutputStream(fileOut)) {
       out.writeObject(person);
       System.out.println("Serialized data is saved in person.ser");

       }
}
```

In the above code, we have successfully serialized the person class and saved it in the **person**.ser file.

And we used **FiltOutputStream** class for sending the class from Java to our local system. And this FiltOutputStream will be fully explained in IO Concepts

What is Deserialization?

Deserialization is the reverse process of serialization. It takes the saved data (bytes) and reconstructs the original object.

Real-World Analogy: **Physical Document:** Continuing the document example, deserialization would be like taking the digital PDF file and converting it back into a readable document format that you can edit or use.

Why Use Deserialization?

Retrieve Data: We can load the object's state back into memory from a file or database.

Receive Data: We can receive objects sent over a network and use them in your program.

How Deserialization and Work in Java

For deserialization we call the readObject() method of *ObjectInputStream* class.

FileInputStream class

Purpose: FileInputStream is used to read data from a file.

How it works: It reads data byte by byte, which makes it useful for reading binary data. However, it can also be used to read text files (though for text, other classes like FileReader might be simpler).

Example: If you want to open a file and read its contents, FileInputStream can do that by reading each byte from the file.

ObjectInputStream class

An ObjectInputStream deserializes objects and primitive data written using an ObjectOutputStream.

Constructor

1) public ObjectInputStream(InputStream in) throws IOException {}	It creates an ObjectInputStream that reads from the specified InputStream.

Important Methods of ObjectInputStream class used in Deseriliazation

Method	Description
1) public final Object readObject() throws IOException, ClassNotFoundException{}	It reads an object from the input stream.
2) public void close() throws IOException {}	It closes ObjectInputStream.

Deserialization Example Code:

To deserialize, we read the `person.ser` file and reconstruct the `Person` object.

```Java
import java.io.FileInputStream;  // For reading the file containing the serialized object
import java.io.ObjectInputStream; // For deserializing the object from the file

public class DeserializeExample {
    public static void main(String[] args) {
```

```
        // Using try-with-resources to automatically close the
streams
        try (FileInputStream fileIn = new
FileInputStream("person.ser"); // Open the file
             ObjectInputStream in = new ObjectInputStream(fileIn)) {
// Create an ObjectInputStream

            // Read the serialized object from the file and cast it
to Person
            Person person = (Person) in.readObject();

            // Display the deserialized object's data
            System.out.println("Deserialized Person:");
            System.out.println("Name: " + person.getName()); //
Assuming getName() is defined in Person
            System.out.println("Age: " + person.getAge());   //
Assuming getAge() is defined in Person

        } catch (Exception e) {
            e.printStackTrace(); // Print the stack trace for any
exceptions that occur
        }
    }
}
```

SerialVersionUID

What It Means: `serialVersionUID` is a unique identifier for a class. It's used to verify the sender and receiver of a serialized object are compatible (i.e., they are using the same version of the class).

Real-World Analogy: **Version Number:** Think of `serialVersionUID` like a document version number. If you send a document with version 1 and try to open it with a version 2 reader, there could be compatibility issues. `serialVersionUID` ensures that the versions match.

Example

```java
import java.io.Serializable;

class Order implements Serializable {
      private static final long serialVersionUID = 1L;
      String item;
      int quantity;

      public Order(String item, int quantity) {
      this.item = item;
      this.quantity = quantity;
      }
}
```

In this example, `serialVersionUID` helps maintain compatibility when the `Order` class changes. If the class definition changes but `serialVersionUID` remains the same, the older version of `Order` can still be deserialized.

Java Serialization with Inheritance (IS-A Relationship)

What It Means: In Java, if a parent class is serializable, then its child classes (subclasses) automatically become serializable too. This is because the child class "is a" type of the parent class.

Real-World Analogy: **Family:** Think of this as a family inheritance. If a parent has certain assets (properties), the child inherits those assets. Similarly, if a parent class is serializable, the child class will inherit this ability.

Example

```Java
import java.io.*; // Importing required classes for serialization

// Vehicle class implements Serializable to allow its objects to be
serialized
class Vehicle implements Serializable {
    String brand; // Brand of the vehicle
    int year;     // Year of manufacture

    // Constructor to initialize the brand and year
    public Vehicle(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }
}

// Car class extends Vehicle and also implements Serializable
class Car extends Vehicle {
    int doors; // Number of doors in the car

    // Constructor to initialize brand, year, and doors
    public Car(String brand, int year, int doors) {
        super(brand, year); // Call the parent class constructor
        this.doors = doors;
    }
}

// Main class to demonstrate serialization and deserialization
public class SerializationTest {
    public static void main(String[] args) throws Exception {
        // Step 1: Create a Car object
        Car car = new Car("Mercedes", 2022, 4);

        // Step 2: Serialize the Car object to a file
        FileOutputStream fos = new FileOutputStream("Vehicle.ser"); // File
output stream
        ObjectOutputStream oos = new ObjectOutputStream(fos);       //
Object output stream
        oos.writeObject(car); // Write the Car object to the file
        oos.close(); // Close the stream
        System.out.println("Car serialized successfully!");

        // Step 3: Deserialize the Car object from the file
        FileInputStream fis = new FileInputStream("Vehicle.ser"); // File
input stream
```

```
        ObjectInputStream ois = new ObjectInputStream(fis);       // Object
input stream
        Car deserializedCar = (Car) ois.readObject(); // Read the Car object
from the file
        ois.close(); // Close the stream

        // Step 4: Display the deserialized Car object's data
        System.out.println("Car deserialized successfully!");
        System.out.println("Brand: " + deserializedCar.brand);
        System.out.println("Year: " + deserializedCar.year);
        System.out.println("Doors: " + deserializedCar.doors);
    }
}
```

Java Serialization with Aggregation (HAS-A Relationship)

What It Means: When a class has a reference to another class (called "HAS-A" relationship), serialization may depend on whether the referenced class is also serializable.

Real-World Analogy: **Library:** Imagine you have a library (`Library` class) that contains many books (`Book` class). If you want to serialize the library, the books also need to be serializable. Otherwise, only the library information (without books) can be serialized.

Example

```Java
class Book implements Serializable {
      String title;

      public Book(String title) {
      this.title = title;
      }
}

class Library implements Serializable {
      String name;
      Book book;
```

```
        public Library(String name, Book book) {
        this.name = name;
        this.book = book;
        }
}
```

Here, `Library` contains a `Book` object. For `Library` to serialize completely, `Book` also needs to implement `Serializable`.

Example

```Java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;

class Book implements Serializable {
        String title;

        public Book(String title) {
        this.title = title;
        }
}

class Library implements Serializable {
        String name;
        Book book;

        public Library(String name, Book book) {
        this.name = name;
        this.book = book;
        }
}

public class SerializationTestIsA {
        public static void main(String[] args) throws Exception {
        Book book = new Book("Java Guide");
```

```
        Library c = new Library("Sukumar", book);

        FileOutputStream fos = new FileOutputStream("Library.ser");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(c);

        FileInputStream fis = new FileInputStream("Library.ser");
        ObjectInputStream ois = new ObjectInputStream(fis);
        Library c1 = (Library) ois.readObject();
        System.out.println(c1.book.title+" "+c1.name);

        }
}
```

Here, `Library` contains a `Book` object. For `Library` to serialize completely, `Book` also needs to implement `Serializable`.

Java Serialization with Static Data Members

What It Means: Static members (variables with `static` keyword) are part of the class itself, not any particular object, so they aren't serialized with the object's state.

Real-World Analogy: **Public Notice:** Think of static members as a shared resource, like a public notice board in a classroom. No matter how many students (objects) there are, the notice board (static member) belongs to the class (school), not the individual students.

Example

```Java
import java.io.Serializable;
class Employee implements Serializable {
        static String company = "TechCorp";
        String name;
```

```
        public Employee(String name) {
        this.name = name;
        }
}
```

In this example, the static `company` field won't be serialized with each `Employee` object. It will retain its value across all instances.

Java Serialization with Arrays

What It Means: Java allows arrays to be serialized if the objects within the array are serializable. This is useful for saving the state of multiple objects at once.

Real-World Analogy: **Group Photo:** Imagine taking a group photo. If each person in the photo (each element in the array) agrees to be photographed (is serializable), the entire group photo (array) can be saved as a file.

Example

```Java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
// Working with arrays in Serializable
class Student implements Serializable {
        String name;

        public Student(String name) {
        this.name = name;
        }
}

class SchoolClass implements Serializable {
        Student[] students;
```

```
        public SchoolClass(Student[] students) {
        this.students = students;
        }

        public void print (Student[] arr) {
        for (int i = 0; i < arr.length; i++) {

              System.out.println(arr[i].name);
        }
        }
}

public class SerilizableArray {
        public static void main(String[] args) throws Exception {
        Student[] student = new Student[3]; // Creating Student Array
        student[0] = new Student("Vishwa");
        student[1] = new Student("Sakib");
        student[2] = new Student("Sukumar");

        SchoolClass school = new SchoolClass(student); // passing
student class array

        // FileOutputStream fos = new FileOutputStream(new
ObjectOutputStream());
        ObjectOutputStream ous = new ObjectOutputStream(new
FileOutputStream("SchoolClass.ser"));
        // Above Inserted FileOutputStream as a constructor argument in
ObjectOutputStream

        ous.writeObject(school); // Serialization Done

        System.out.println("SchoolClass Serialized");

        ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("SchoolClass.ser"));
        SchoolClass school1 = (SchoolClass)ois.readObject(); // Casted
to SchoolClass

        school1.print(school.students); // Array of an SchoolClass
        }
}
```

In this example, `SchoolClass` contains an array of `Student` objects. Since each `Student` is serializable, the array can be serialized as part of the `SchoolClass`.

Externalizable in Java

What It Means: The `Externalizable` interface is an advanced alternative to `Serializable`. It allows you to manually control how an object is serialized and deserialized by implementing two methods: `writeExternal` and `readExternal`.

Real-World Analogy: Think of `Externalizable` as a custom recipe. Instead of following a standard recipe (default serialization), you decide how to save and read each ingredient yourself.

Example

```Java
import java.io.*;

class Product implements Externalizable {
      String name;
      int price;

      public Product() {} // Mandatory no-arg constructor

      public Product(String name, int price) {
      this.name = name;
      this.price = price;
      }

      @Override
      public void writeExternal(ObjectOutput out) throws IOException
{
      out.writeObject(name);
      out.writeInt(price);
      }
```

```
        @Override
        public void readExternal(ObjectInput in) throws IOException,
ClassNotFoundException {
        name = (String) in.readObject();
        price = in.readInt();
        }
}

public class ExternalizationTest {
        public static void main(String[] args) throws Exception {
        Product p = new Product("IPhone", 60000);
        ObjectOutputStream ous = new ObjectOutputStream(new
FileOutputStream("Product.ser")); // Object Searilized
        ous.writeObject(p); // Converted into bytecode

        System.out.println("Product class Searilized");

        ObjectInputStream ois =  new ObjectInputStream(new
FileInputStream("Product.ser")); // Deserialized
        Product p1 = (Product) ois.readObject(); // Reading the Object
and Object downcasted to Product

        System.out.println("Product class Desearilized");

        System.out.println(p1.name+" "+p1.price);
        }
}
```

Here, `Product` is an externalizable class. You explicitly control how `name` and `price` are written to and read from the stream.

Output

```
Java
Product class Serialized
Product class Deserialized
IPhone 60000 ABC1234
```

Java Transient Keyword

What It Means: The `transient` keyword can be used to mark a variable that you don't want to include in serialization. This is useful for sensitive data like passwords.

Real-World Analogy: **Do Not:** Think of `transient` as a do-not-photograph sign. In a group photo, if one person (variable) has the sign, they won't appear in the final photo (serialized output).

Example

```Java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;

class User implements Serializable {
      String username;
      transient String password; // Won't be serialized

      public User(String username, String password) {
      this.username = username;
      this.password = password;
      }
}

public class TrancientTest {
      public static void main(String[] args) throws Exception {
      User user = new User("Viswa", "ABCD12345");
      ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("User.ser"));
      oos.writeObject(user);
```

```
        System.out.println("User class Searilized");

        ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("User.ser"));
        User user2 = (User) ois.readObject();

        System.out.println(user2.username+" "+user2.password);

        }
}
```

In this example, `password` is marked `transient`, so it won't be serialized. When the `User` object is deserialized, `password` will be `null`.

Output

```
Java
User class Searilized

Viswa null // Because it it made trainsient using transient keyword
```

This confirms that `name` and `price` were saved and restored correctly, and the `password` was modified as specified.

Custom Serialization in Java

What It Means: Custom serialization allows you to control exactly how each field of an object is serialized and deserialized. Normally, Java's `Serializable` interface handles the process automatically, but with custom serialization, you can define which fields to save and even modify them before serialization. This is useful for handling sensitive data or optimizing object storage.

How It Works

To implement custom serialization, a class can define two special methods:

- ➔ **`private void writeObject(ObjectOutputStream out)`**: This method is used to customize how data is written during serialization.
- ➔ **`private void readObject(ObjectInputStream in)`**: This method customizes how data is read during deserialization.

These methods let you control what is saved, what is ignored, or even transform certain fields during the serialization process.

Real-World Analogy for Custom Serialization: Imagine you're mailing a document that contains sensitive information. Before sending it, you choose to black out specific parts of the document. When the recipient opens it, they see the hidden information but not the original content. Similarly, custom serialization lets you hide or transform certain parts of the data before it's saved and then reconstruct it during retrieval.

Example Use Case: Secure Password Storage

Suppose you're creating a `User` class where the password should be saved in an encrypted form instead of plain text. Here's how custom serialization can help:

Code Example

```Java
import java.io.*;
class User implements Serializable {
      private static final long serialVersionUID = 1L;
      private String username;
      private transient String password; // Marked transient to
exclude from default serialization

      public User(String username, String password) {
      this.username = username;
```

```
        this.password = password;
        }

        // Custom writeObject method for serialization
        private void writeObject(ObjectOutputStream out) throws
IOException {
        out.defaultWriteObject(); // Serialize non-transient fields
(like username)

        // Encrypt password before serialization
        String encryptedPassword = "ENCRYPTED_" + password; //
Simplified encryption example
        out.writeObject(encryptedPassword);
        }

        // Custom readObject method for deserialization
        private void readObject(ObjectInputStream in) throws
IOException, ClassNotFoundException {
        in.defaultReadObject(); // Deserialize non-transient fields

        // Read and decrypt password after deserialization
        String encryptedPassword = (String) in.readObject();
        this.password = encryptedPassword.replace("ENCRYPTED_", ""); //
Simplified decryption
        }

        @Override
        public String toString() {
        return "Username: " + username + ", Password: " + password;
        }
}

public class CustomSerializationExample {
        public static void main(String[] args) {
        // Serialize the User object
        ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("user.ser")));
                User user = new User("john_doe", "mypassword123");
                out.writeObject(user);
                System.out.println("User serialized: " + user);
        }

        // Deserialize the User object
```

```
        ObjectInputStream in = new ObjectInputStream(new
FileInputStream("user.ser"));
            User deserializedUser = (User) in.readObject();
            System.out.println("User deserialized: " +
deserializedUser);
        }
        }
}
```

Explanation of the Code

- ➔ **Transient Keyword**: The `password` field is marked `transient`, meaning it will be ignored during the default serialization process.
- ➔ **Custom `writeObject` Method**: Calls `out.defaultWriteObject()` to serialize all non-transient fields.
- ➔ Encrypts the `password` field (e.g., adds a prefix `"ENCRYPTED_"`).
- ➔ Writes the encrypted password to the output stream.
- ➔ **Custom `readObject` Method**: Calls `in.defaultReadObject()` to deserialize all non-transient fields.
- ➔ Reads the encrypted password, removes the prefix to "decrypt" it, and assigns it back to `password`.

When to Use Custom Serialization

- ➔ **Sensitive Information**: For storing fields like passwords, credit card numbers, or personal data securely.
- ➔ **Data Transformation**: When you want to save a modified version of a field (e.g., compressing data before saving).
- ➔ **Optimized Storage**: For large objects where you might want to skip serializing fields that don't need to be stored every time.

Custom Serialization in Java Using Externalization

Achieved by Overriding inbuilt methods of Externalize Interface

Example

```java
import java.io.*;

class Product implements Externalizable {
      String name;
      int price;
      String password = "ABC1234";

      public Product() {} // Mandatory no-arg constructor

      public Product(String name, int price) {
      this.name = name;
      this.price = price;
      }

      @Override
      public void writeExternal(ObjectOutput out) throws IOException
{
      out.writeObject(name);
      out.writeInt(price);
      out.writeObject(password+"345");
      }

      @Override
      public void readExternal(ObjectInput in) throws IOException,
ClassNotFoundException {
      name = (String) in.readObject();
      price = in.readInt();
      String reWrite = (String) in.readObject();
      password = reWrite.replace("345", "");
      }
}

public class ExternalizationTest {
```

```
        public static void main(String[] args) throws Exception {
        Product p = new Product("IPhone", 60000);
        ObjectOutputStream ous = new ObjectOutputStream(new
FileOutputStream("Product.ser")); // Object Searilized
        ous.writeObject(p); // Converted into bytecode

        System.out.println("Product class Searilized");

        ObjectInputStream ois =  new ObjectInputStream(new
FileInputStream("Product.ser")); // Deserialized
        Product p1 = (Product) ois.readObject(); // Reading the Object
and Object downcasted to Product

        System.out.println("Product class Desearilized");

        System.out.println(p1.name+" "+p1.price+" "+p1.password);

        }
}
```

This confirms that name and price were saved and restored correctly, and the password was modified as specified.

Code Explanation:

Product Class:

- ➔ Contains three fields: `name`, `price`, and `password`.
- ➔ Defines two constructors: a no-argument constructor (mandatory for `Externalizable` classes) and a parameterized constructor.
- ➔ Implements two methods from the `Externalizable` interface:
- ➔ `writeExternal(ObjectOutput out)`: Used to specify which data to write out. In this case, it writes the `name`, `price`, and `password` (with "345" appended).
- ➔ `readExternal(ObjectInput in)`: Used to specify how to read the serialized data back into the object. Here, `name` and `price`

are restored, and `password` is read and trimmed back to "ABC1234".

ExternalizationTest Class:

Creates an instance of `Product` with specific values for `name` and `price`.

Uses an `ObjectOutputStream` to serialize (write) the `Product` object to a file named "Product.ser".

Deserializes (reads) the object from "Product.ser" using an `ObjectInputStream`, effectively recreating the `Product` object as `p1`.

Displays the restored values of `name`, `price`, and `password` to verify the custom deserialization.

Garbage Collection in Java

Garbage Collection (GC) in Java is the process through which Java automatically identifies and frees up memory by removing objects that are no longer being used by the program. This helps in optimizing memory usage and enhances the performance of Java applications by preventing memory leaks and keeping the application efficient.

Key Features of Garbage Collection

Automatic Memory Management
Java's garbage collection is automatic, meaning developers don't need to manually free memory, unlike languages like C/C++. The JVM (Java Virtual Machine) handles it.

Real-world Analogy: Imagine your room being automatically cleaned up by a robot vacuum cleaner whenever there's clutter (old

objects you no longer need). You don't need to do the cleaning yourself.

Heap Memory Allocation
Java objects are stored in a memory area called the heap. Garbage collection focuses on reclaiming memory from the heap by removing unused objects.

Real-world Analogy: The heap memory is like a storage room in your house. Garbage collection works to make space in that room by removing items you no longer use.

Phases of Garbage Collection

Garbage Collection has different phases or steps:

Mechanisms Involved in Garbage Collection

Mark and Sweep Algorithm

Marking Phase: The garbage collector scans all reachable objects (those that can still be used by the application) and "marks" them.

Sweeping Phase: It then goes through the heap, deletes any objects that aren't marked as reachable, freeing up memory.

Compacting: After sweeping, the GC compacts memory by shifting objects together, so there is a continuous memory block for new objects.

Real-world Analogy: Think of marking as you identifying items you still need in a storage room. Sweeping is like throwing away items you don't use, and compacting is arranging the remaining items in an organized way.

Generational Garbage Collection

The heap is divided into sections (or generations) that store objects based on their life cycle:

Young Generation: For new objects. If they survive several rounds of GC, they move to the Old Generation.

Old Generation: For longer-lasting objects.

Permanent Generation (PermGen): Stores metadata and static information.

Real-world Analogy: Imagine your home is organized into rooms. You put new items in one room (Young Generation). If they're useful for a longer time, you move them to another room (Old Generation). Permanent items, like family photos, go into a separate storage space (PermGen).

Garbage Collection Types in Java

Minor GC: Cleans up the Young Generation. It's quick because most objects in this area are short-lived.

Major GC: Cleans the Old Generation. It takes longer since it involves more data.

Full GC: Cleans both Young and Old Generations and may include PermGen if needed. It's the most time-consuming.

Real-World Example for Garbage Collection Phases

Minor GC: Like picking up scattered toys in a playroom. It's quick since you're focusing on small items (short-lived objects in Young Gen).

Major GC: Like cleaning an entire attic with old stuff that's been there for a while. It takes more time because of the quantity and age of items.

Full GC: This is like cleaning your whole house, including every room and storage space. It's the most comprehensive clean-up but takes the most time.

Real-time Usage and Examples

Simple Example of Garbage Collection Trigger

```Java
public class GarbageCollectionExample {
      public static void main(String[] args) {
      GarbageCollectionExample obj = new
GarbageCollectionExample();
      obj = null; // making obj eligible for GC

      // Requesting GC
      System.gc();
      System.out.println("Garbage Collection requested.");
      }

      @Override
      protected void finalize() {
      System.out.println("Garbage Collector has freed up the
memory.");
      }
}
```

Explanation: Here, obj is set to null, meaning it no longer points to any memory location, making it eligible for garbage collection. The System.gc() method hints that garbage collection should occur.

Advanced Example with Generational Collection

```Java
public class AdvancedGCExample {
      public static void main(String[] args) {
      for (int i = 0; i < 10000; i++) {
            String temporary = "temp" + i;
            // Objects in Young Generation
```

```
        }

        String longLived = "This is a long-lived object";
        System.gc(); // request GC
        }
    }
```

Explanation: In this example, temporary objects are created and quickly discarded (making them candidates for Minor GC in the Young Generation). The longLived object persists longer, potentially moving to the Old Generation if it stays in use.

Why is Garbage Collection Important?

Memory Efficiency: By removing unused objects, GC keeps memory usage low, which is crucial for apps that need to perform well, like banking apps, e-commerce websites, and high-traffic web applications.

No Memory Leaks: With garbage collection, memory that's no longer needed is automatically cleared, preventing memory from being used up by unnecessary objects.

Developer Ease: Java's automatic memory management removes the developer's need to track and manually free memory.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan